HCC Performance Optimization of ClickHouse for Low CPU Utilization queries in ClickBench

Jiebin Sun (jiebin.sun@intel.com)

From Linux Performance & Innovation, Intel Shanghai

Agenda

- Problem statement and performance results
- Clickbench overviews
- Query 5 analysis and optimization
- Query 42 analysis and optimization

Problem Statement & Optimization

- Request to investigate core scaling issue for Phoronix ClickBench.
  - From 77.1% to 92.6% with optimizations (288c / 144c)

Geometric Mean QPS over 43 ClickBench queries		
Config	**144c**	**288c**
opt/base	111.6%	134.1%

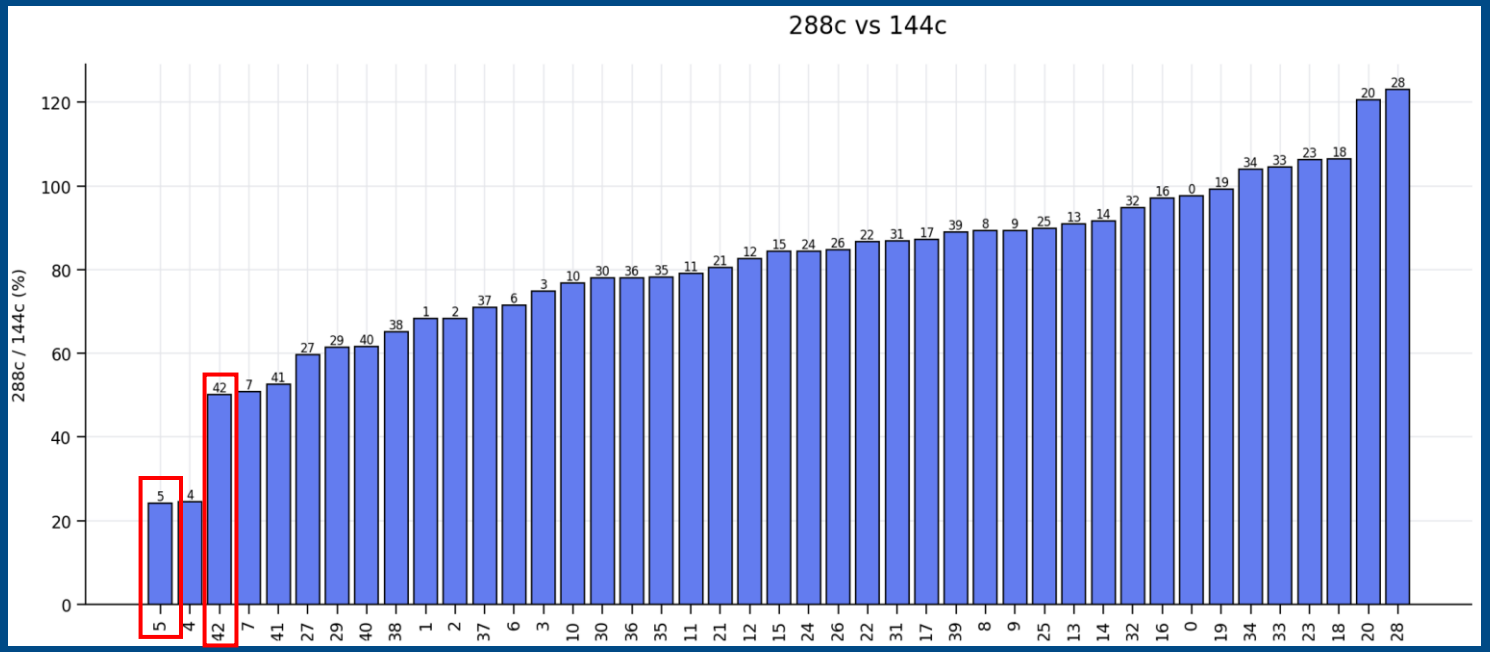
ClickBench Benchmark

- ClickBench: a Benchmark For Analytical Databases
  - Represents typical workloads: clickstream and traffic analysis, web analytics ...
  - Dataset size: nearly 100M records
  - 43 queries in total
  - Metrics: throughput (QPS)
  - Server/Client mode

Query to analysis

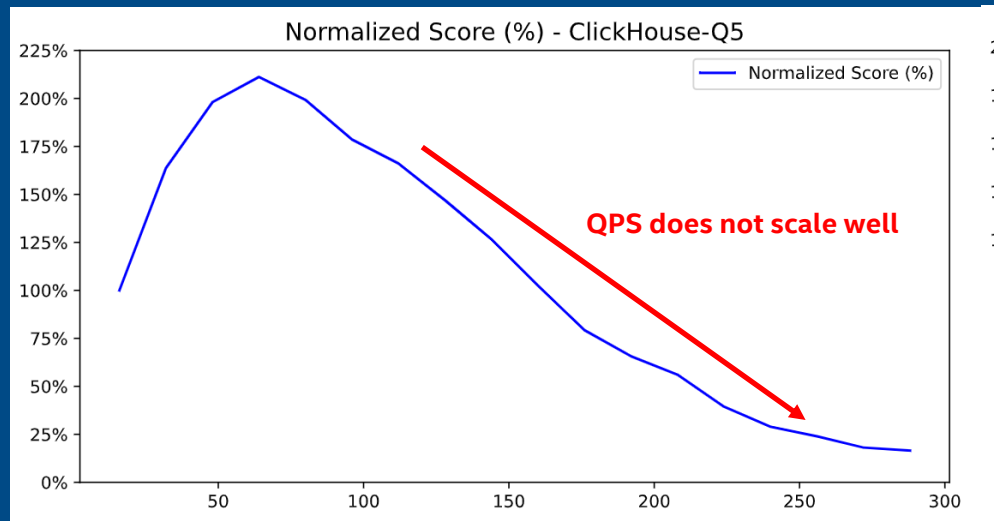
288c / 144c performance ratio

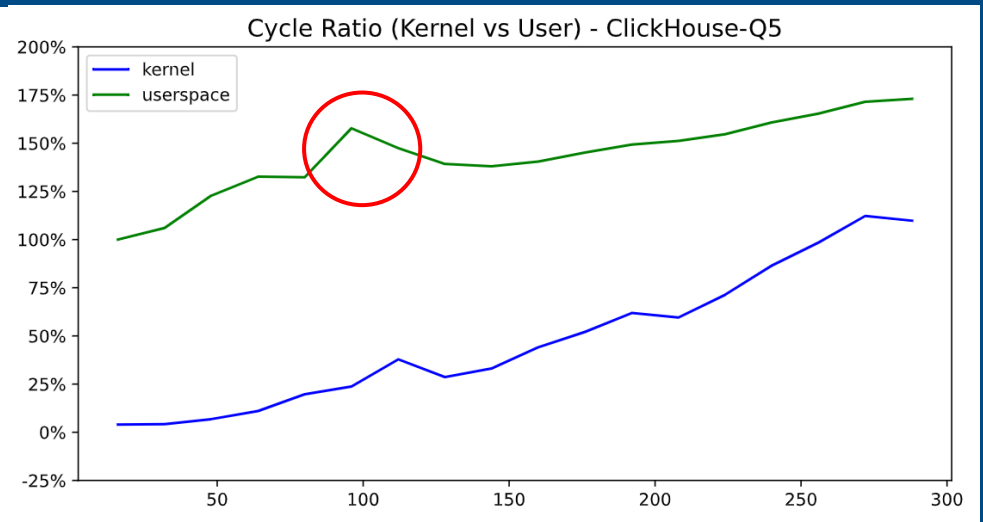
- Select Q5 and Q42 to analyze.
  - Q5: SELECT COUNT(DISTINCT SearchPhrase) FROM hits;
  - Q42: SELECT DATE_TRUNC('minute', EventTime) AS M, COUNT(*) AS PageViews FROM hits WHERE CounterID = 62 AND EventDate >= '2013-07-14' AND EventDate <= '2013-07-15' AND IsRefresh = 0 AND DontCountHits = 0 GROUP BY DATE_TRUNC('minute', EventTime) ORDER BY DATE_TRUNC('minute', EventTime) LIMIT 10 OFFSET 1000;

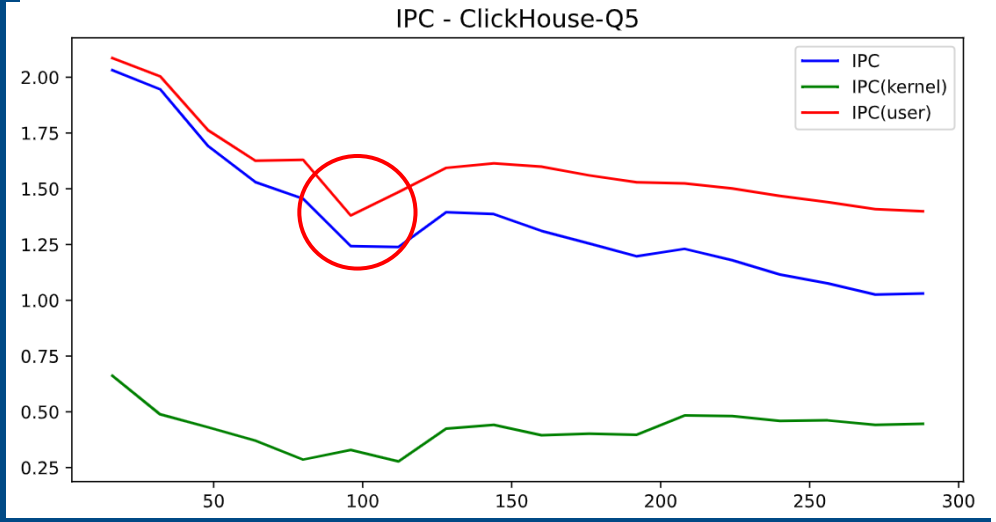


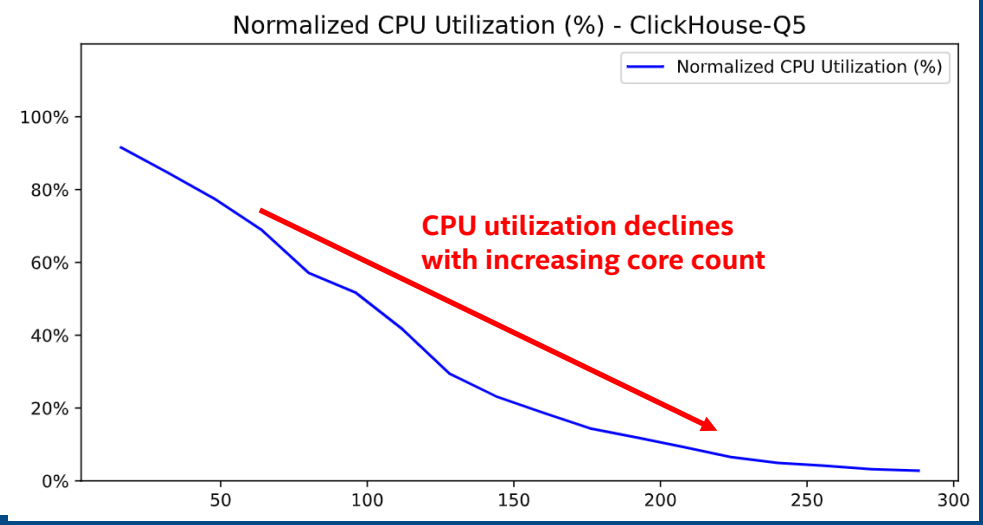
Q5 288c: Core Scaling

Q5: SELECT COUNT(DISTINCT SearchPhrase) FROM hits;



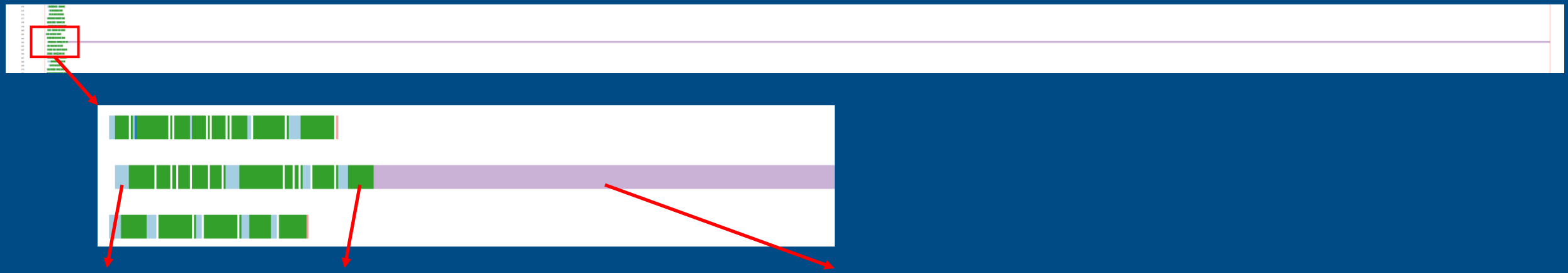




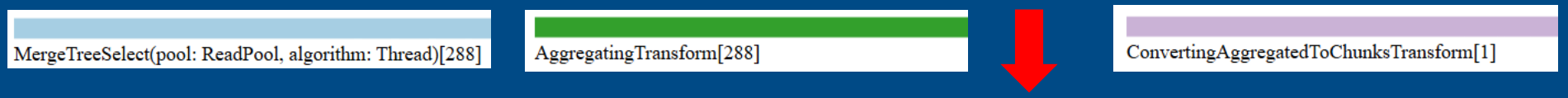


Q5 pipeline

Q5: SELECT COUNT(DISTINCT SearchPhrase) FROM hits;



1. Read data → 2. Aggregate into a two-level hash table → 3. Merge all hash tables



Q5 Pipeline Wall Time Comparison (ms)

Stage	288 Cores	144 Cores	Δ (288c – 144c)	Difference (%)
Total Time	1605.08	234.49	1370.59	100.0%
ConvertingAggregatedToChunksTransform	1580.25	192.14	1388.11	101.3%
Read & Aggregate	24.83	42.35	−17.52	−1.3%

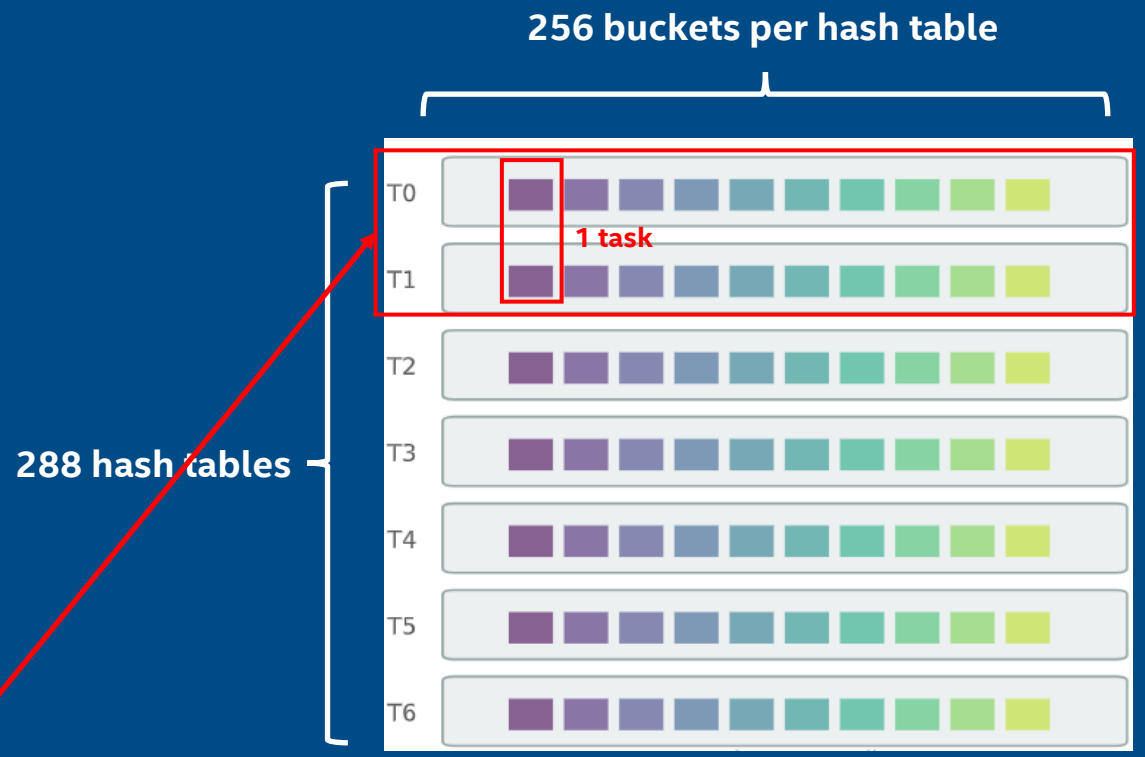
Q5 hotspot

Q5: SELECT COUNT(DISTINCT SearchPhrase) FROM hits;

ConvertingAggregatedToChunksTransform[1]

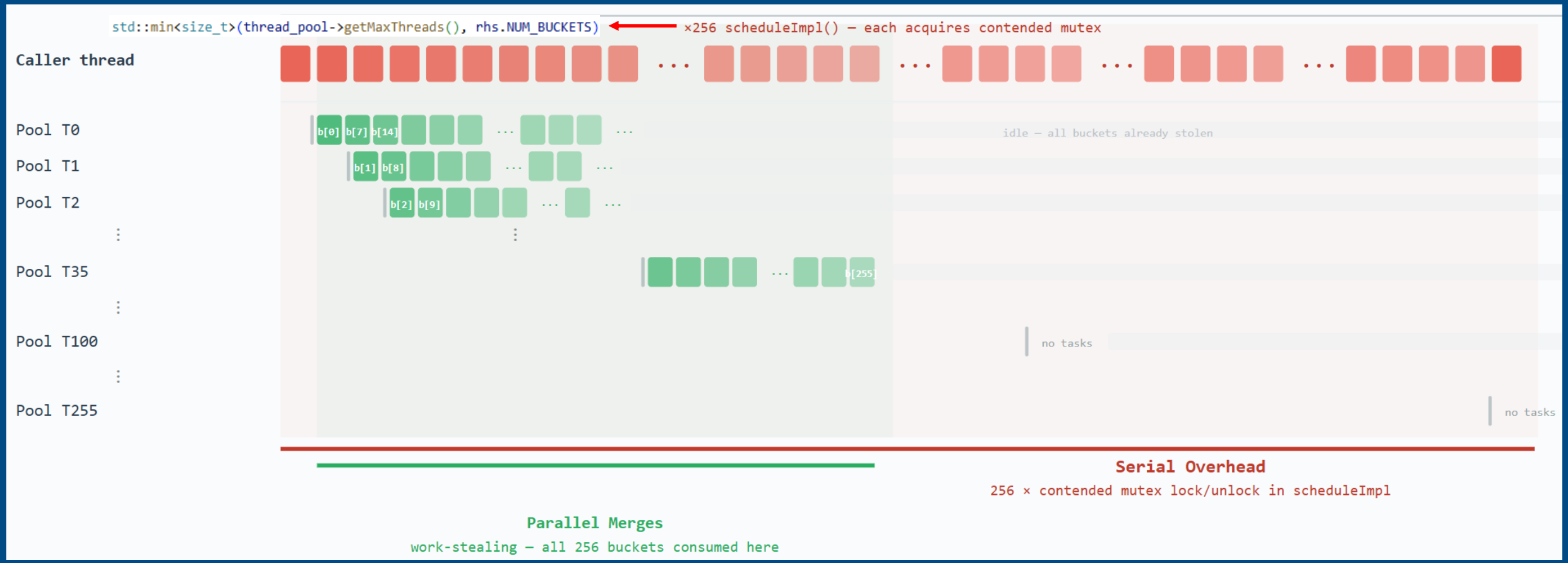
```
ConvertingAggregatedToChunksTransform::work()
  └─ ConvertingAggregatedToChunksWithMergingSource::generate()
       └─ Aggregator::mergeAndConvertOneBucketToChunk()
            └─ Aggregator::mergeBucketImpl<Method>()
                 └─ Aggregator::mergeDataImpl<Method>()
                      └─ AggregateFunctionUniq::merge()
                           └─ UniqExactSet::merge()
                                └─ HashSetTable::merge()
```

UniqExactSet::merge() to merge two two-level hash tables



Q5: SELECT COUNT(DISTINCT SearchPhrase) FROM hits;

Q5 merge two two-level hash tables



Q5 problem statement

Q5: SELECT COUNT(DISTINCT SearchPhrase) FROM hits;

- The task queue is typically exhausted once about 30–40 threads are spawned during each two hash tables merge; additional threads contribute only overhead rather than meaningful parallelism.
- `scheduleImpl` executes serially under a mutex. The heavy reliance and using on this critical section significantly degrades performance and limits scalability.

```
std::min<size_t>(thread_pool->getMaxThreads(), rhs.NUM_BUCKETS)
```

256

	288c	144c	288c / 144c
Total hash tables	288	144	**2.00×**
Spawned threads per round	256	144	**1.78×**
Critical-section calls (total)	73,472	20,592	**3.57×**

Q5 optimization1

Q5: SELECT COUNT(DISTINCT SearchPhrase) FROM hits;

2.98x @Q5 on 288c

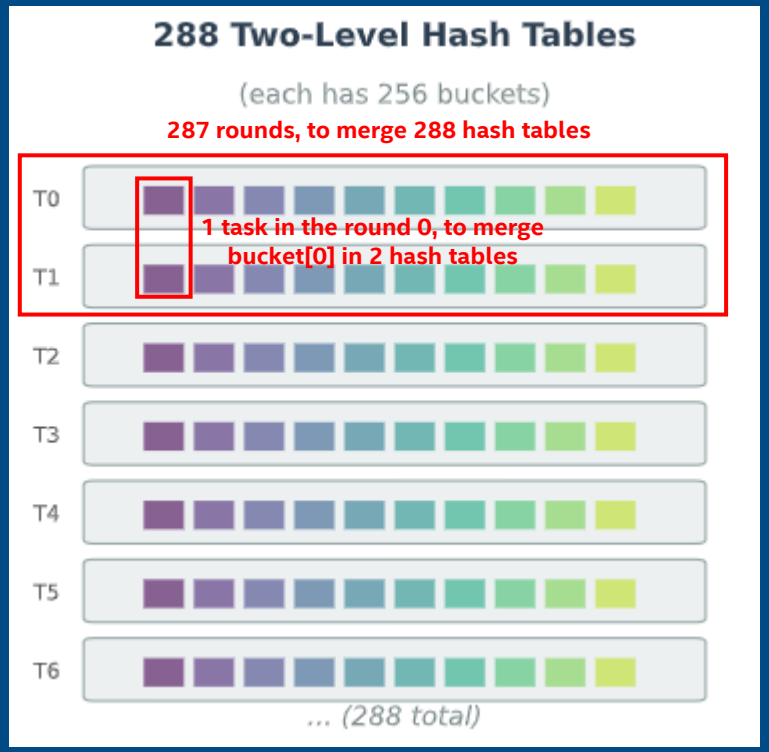
Patch 1: Avoid Redundant Thread Spawning

0001-avoid-redundant-thread-spawning-in-UniqExactSet-merge

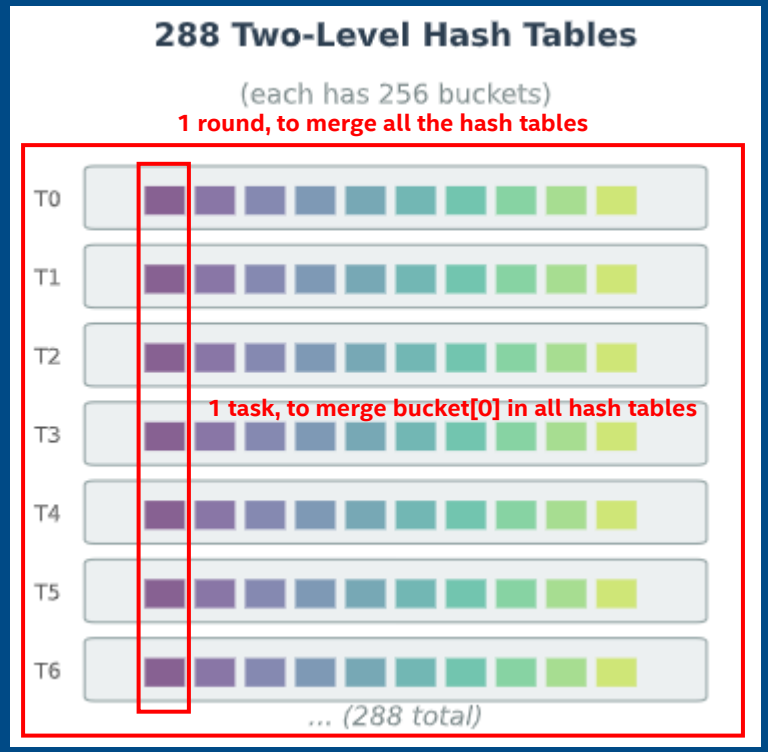
```
// UniqExactSet.h - BEFORE (getMaxThreads called EVERY iteration):
for (size_t i = 0; i < std::min(thread_pool->getMaxThreads(), NUM_BUCKETS); ++i)
    runner.enqueueAndKeepTrack(thread_func);

// UniqExactSet.h - AFTER (cached + early exit):
const size_t max_threads_to_enqueue = std::min(thread_pool->getMaxThreads(), NUM_BUCKETS);
for (size_t i = 0; i < max_threads_to_enqueue
     && next_bucket_to_merge->load(relaxed) < NUM_BUCKETS; ++i)
    runner.enqueueAndKeepTrack(thread_func);
```

Q5 optimization2

Q5: SELECT COUNT(DISTINCT SearchPhrase) FROM hits;



14.9x @Q5 on 288c




```
std::min<size_t>(thread_pool->getMaxThreads(), rhs.NUM_BUCKETS)
```

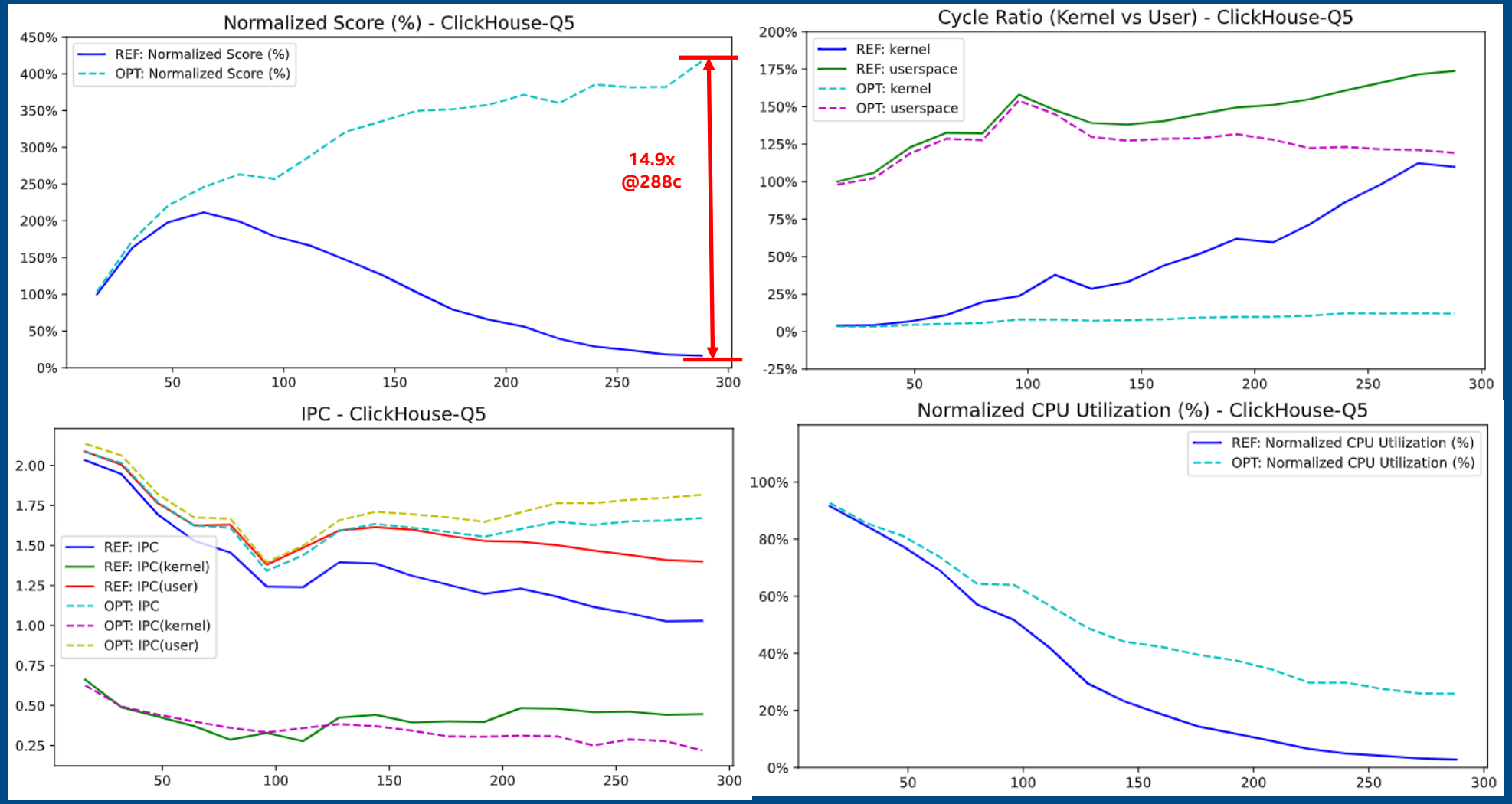
256

288c	base	opt	opt /base
Rounds	287	1	1/287
Spawned threads per round	256	256	1x
Critical-section calls (total)	73,472	256	1/287

opt	288c	144c	288c/144c
Rounds	1	1	1x
Spawned threads per round	256	144	1x
Critical-section calls (total)	256	144	1.78x

Q5 288c: Core Scaling

Q5: SELECT COUNT(DISTINCT SearchPhrase) FROM hits;



Q5: analyze Critical-Section Beyond Invocation Reduction

Q5: SELECT COUNT(DISTINCT SearchPhrase) FROM hits;

- Both Optimization 1 and Optimization 2 reduced the number of critical-section (CS) invocations in ClickBench Q5, helping to mitigate software core scaling limitations.
- We next shifted our focus to analyze the performance of the critical section itself, independent of invocation frequency.

Q5 bpftrace of Critical-Section

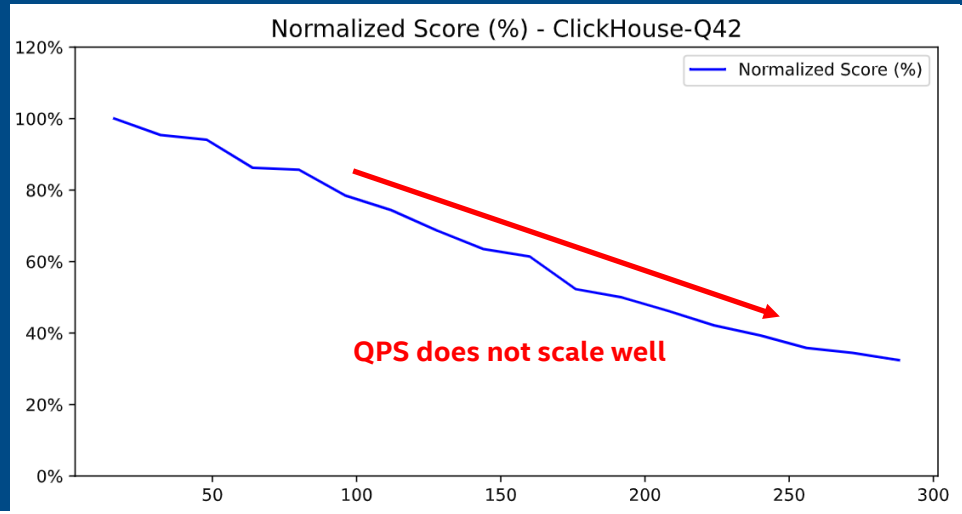
Q5: SELECT COUNT(DISTINCT SearchPhrase) FROM hits;

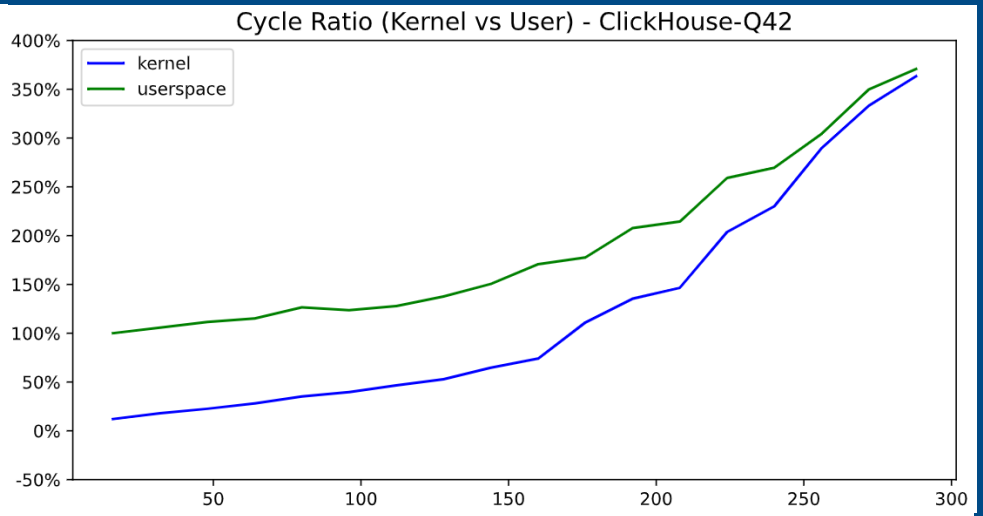
- Using bpftrace, we examined the serial critical section in Q5.

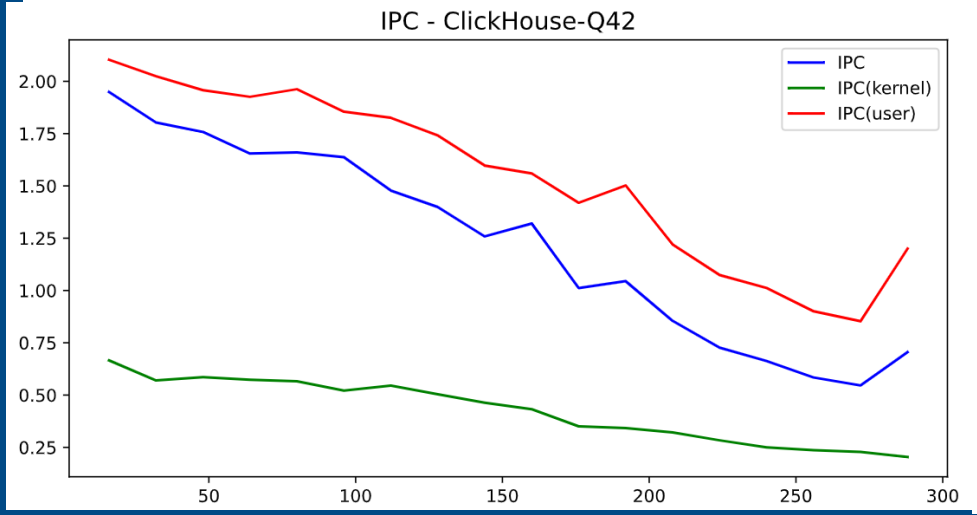
Critical Section (CS) Analysis
Metric
CS count
CS total duration (ms)
CS avg duration (μs)
IPC in CS
CPU frequency in CS (GHz)
QPS

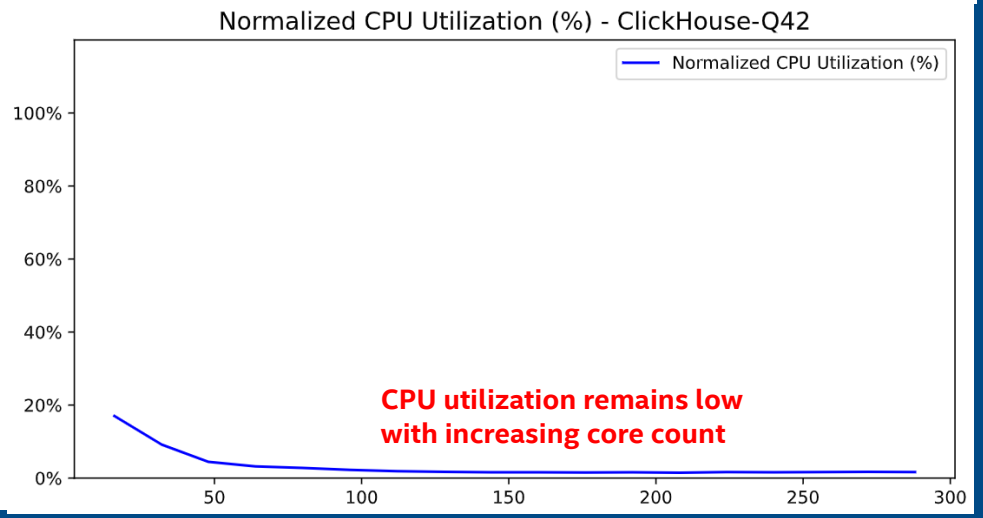
Q42 288c: Core Scaling

Q42: SELECT DATE_TRUNC('minute', EventTime) AS M, COUNT(*) AS PageViews FROM hits WHERE CounterID = 62 AND EventDate >= '2013-07-14' AND EventDate <= '2013-07-15' AND IsRefresh = 0 AND DontCountHits = 0 GROUP BY DATE_TRUNC('minute', EventTime) ORDER BY DATE_TRUNC('minute', EventTime) LIMIT 10 OFFSET 1000;









Q42 off-CPU time

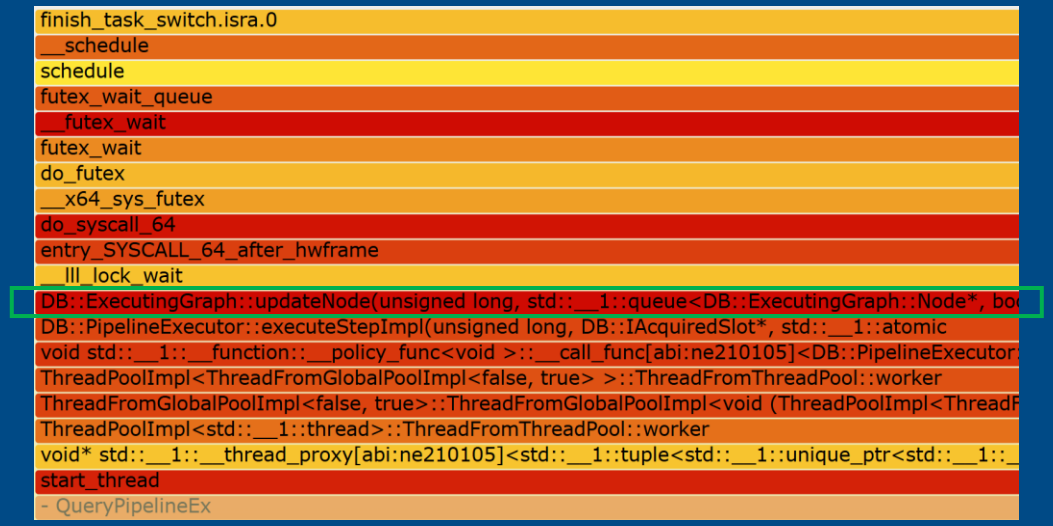
Q42: SELECT DATE_TRUNC('minute', EventTime) AS M, COUNT(*) AS PageViews FROM hits WHERE CounterID = 62 AND EventDate >= '2013-07-14' AND EventDate <= '2013-07-15' AND IsRefresh = 0 AND DontCountHits = 0 GROUP BY DATE_TRUNC('minute', EventTime) ORDER BY DATE_TRUNC('minute', EventTime) LIMIT 10 OFFSET 1000;

Configuration	CPU Utilization
288 cores	2.5%
144 cores	2.8%

Quite low CPU utilization

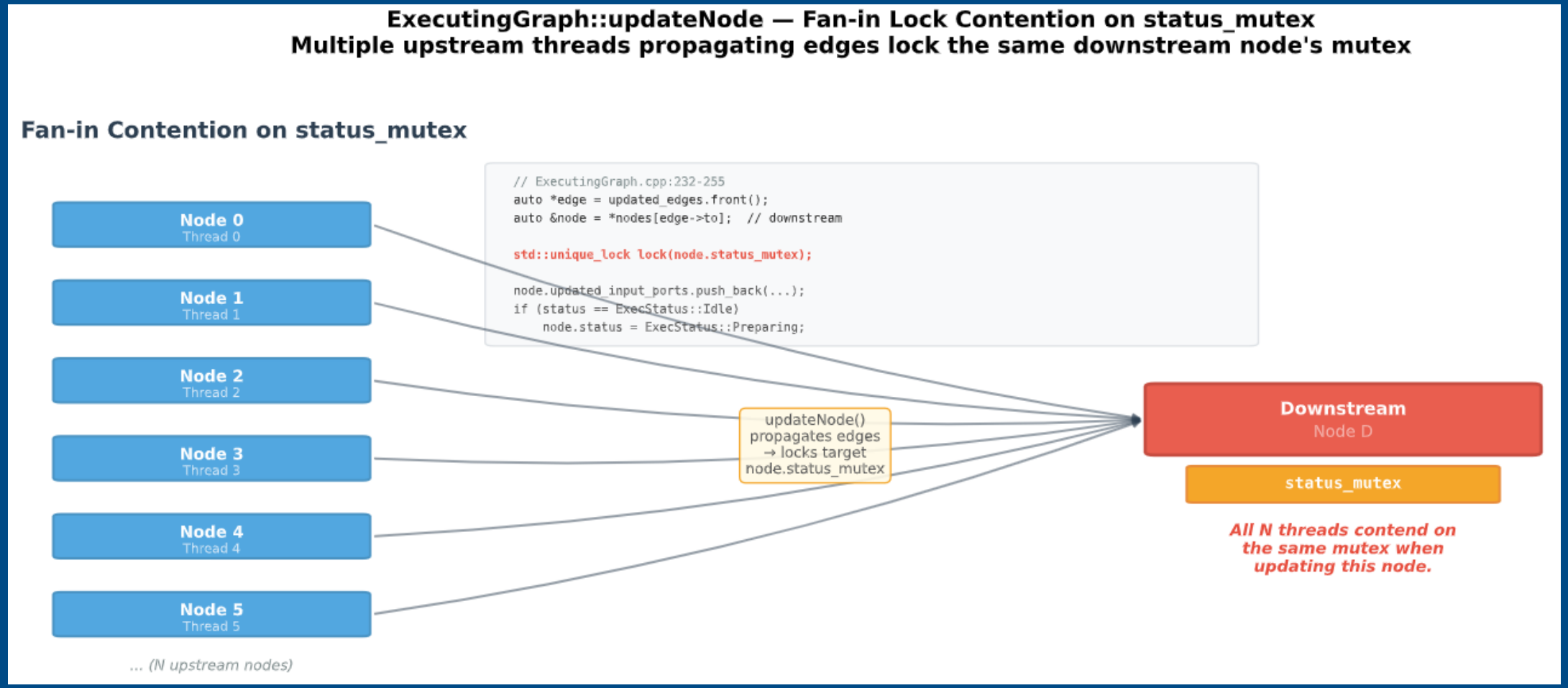
	144 cores	288 cores	Ratio
__lll_lock_wait in updateNode	28,840,850 (1.71%)	165,582,907 (5.86%)	5.74x
Per core	200,284	574,940	2.87x

`__lll_lock_wait`: The primary difference to off-CPU time



Call stack of `__lll_lock_wait` in flame graph

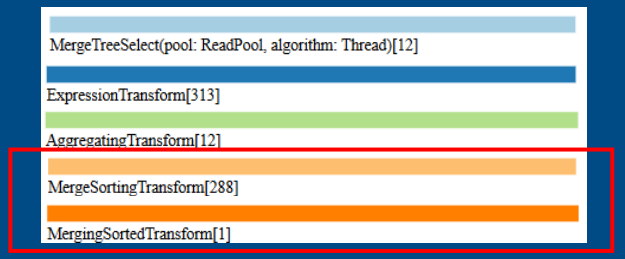
Q42 updateNode

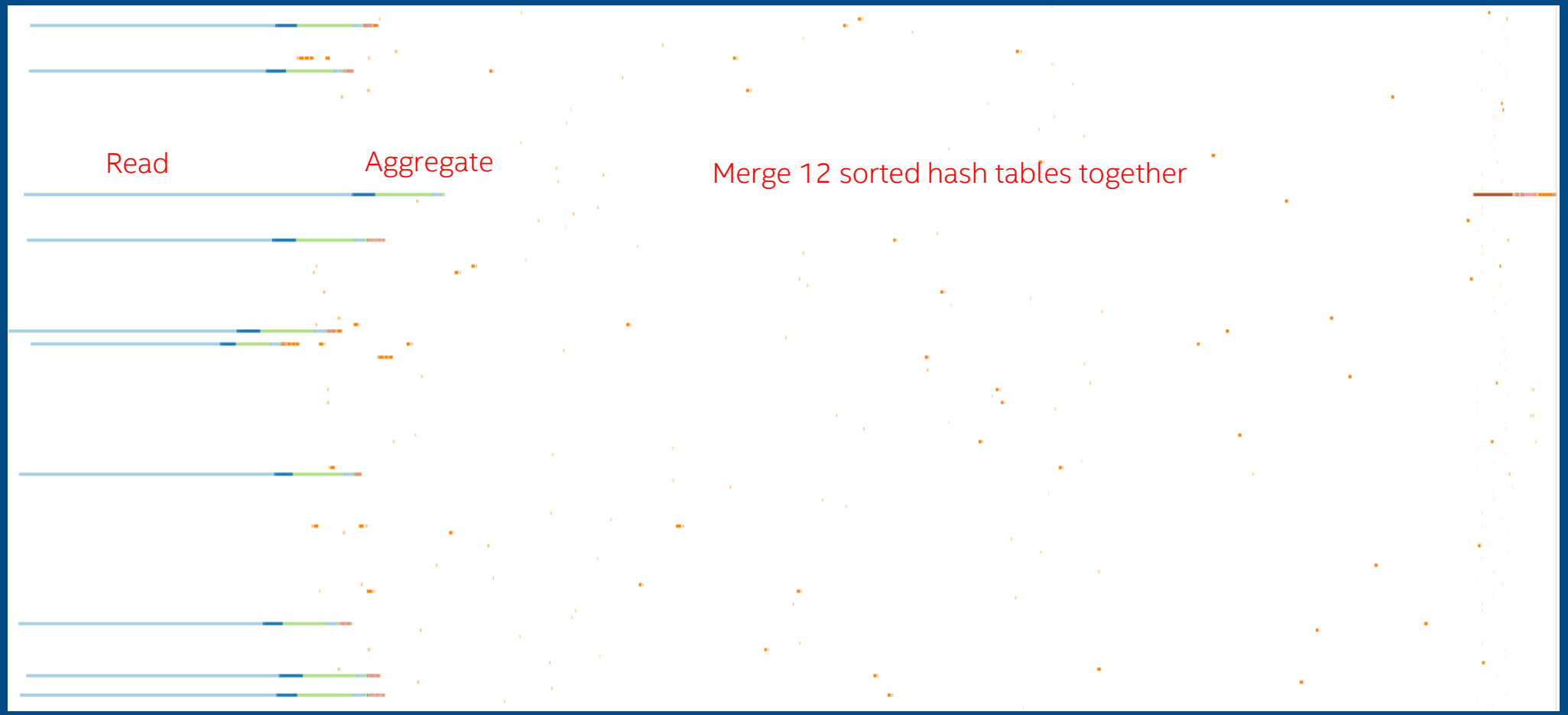


Then which node's status_mutex causes high lock contention?

Q42 pipeline

Q42: SELECT DATE_TRUNC('minute', EventTime) AS M, COUNT(*) AS PageViews FROM hits WHERE CounterID = 62 AND EventDate >= '2013-07-14' AND EventDate <= '2013-07-15' AND IsRefresh = 0 AND DontCountHits = 0 GROUP BY DATE_TRUNC('minute', EventTime) ORDER BY DATE_TRUNC('minute', EventTime) LIMIT 10 OFFSET 1000;





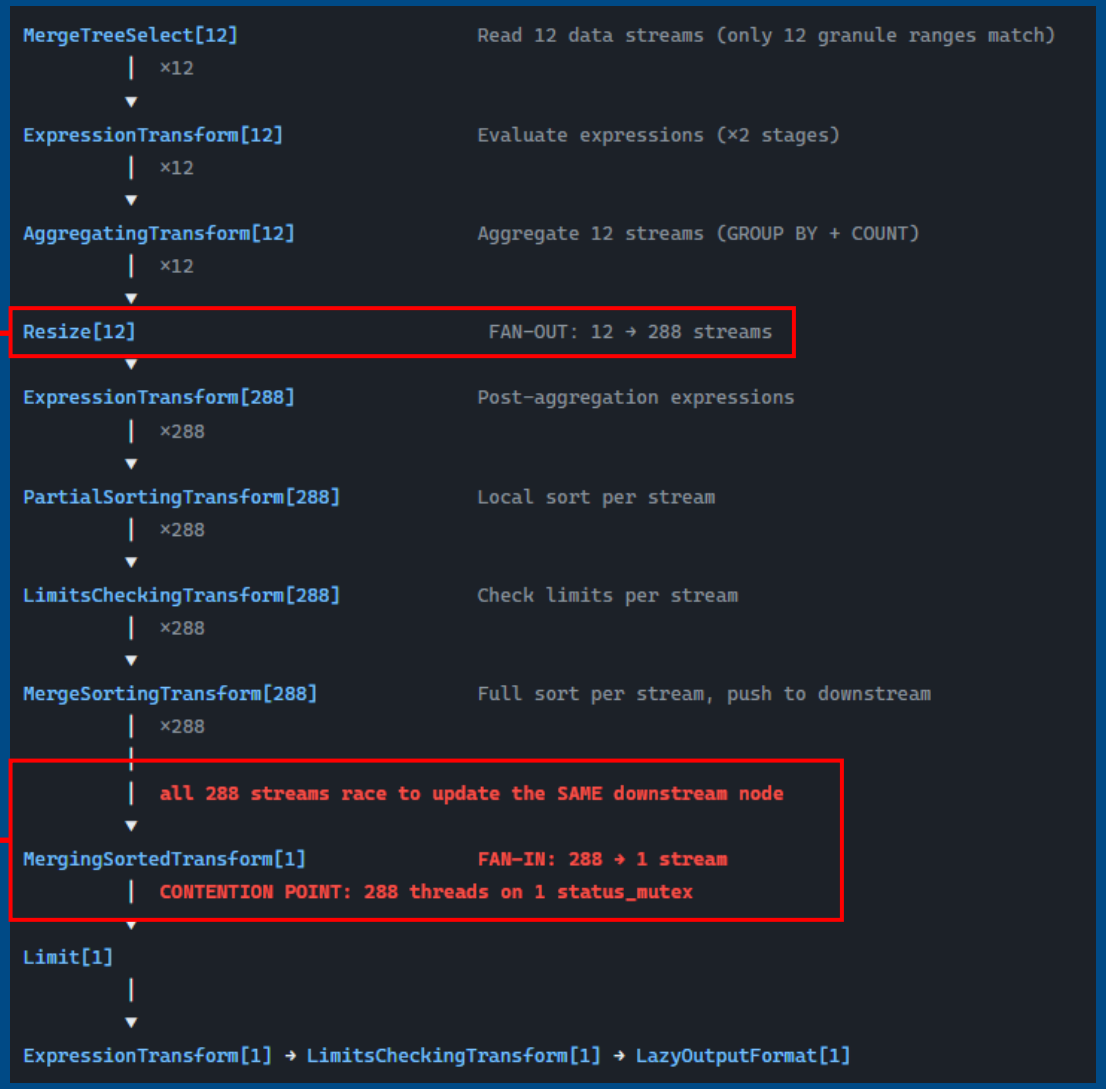
Merge sorted hash tables is the bottle neck

Q42 pipeline

Q42: SELECT DATE_TRUNC('minute', EventTime) AS M, COUNT(*) AS PageViews FROM hits WHERE CounterID = 62 AND EventDate >= '2013-07-14' AND EventDate <= '2013-07-15' AND IsRefresh = 0 AND DontCountHits = 0 GROUP BY DATE_TRUNC('minute', EventTime) ORDER BY DATE_TRUNC('minute', EventTime) LIMIT 10 OFFSET 1000;

After aggregation, it fans out from these 12 nodes to streams matching the full thread count to complete the remaining sorting and limiting operations.

Heavy lock contention happens when merging 288 nodes to single node.



Q42 optimizations

Q42: SELECT DATE_TRUNC('minute', EventTime) AS M, COUNT(*) AS PageViews FROM hits WHERE CounterID = 62 AND EventDate >= '2013-07-14' AND EventDate <= '2013-07-15' AND IsRefresh = 0 AND DontCountHits = 0 GROUP BY DATE_TRUNC('minute', EventTime) ORDER BY DATE_TRUNC('minute', EventTime) LIMIT 10 OFFSET 1000;

2.25x @Q42 on 288c

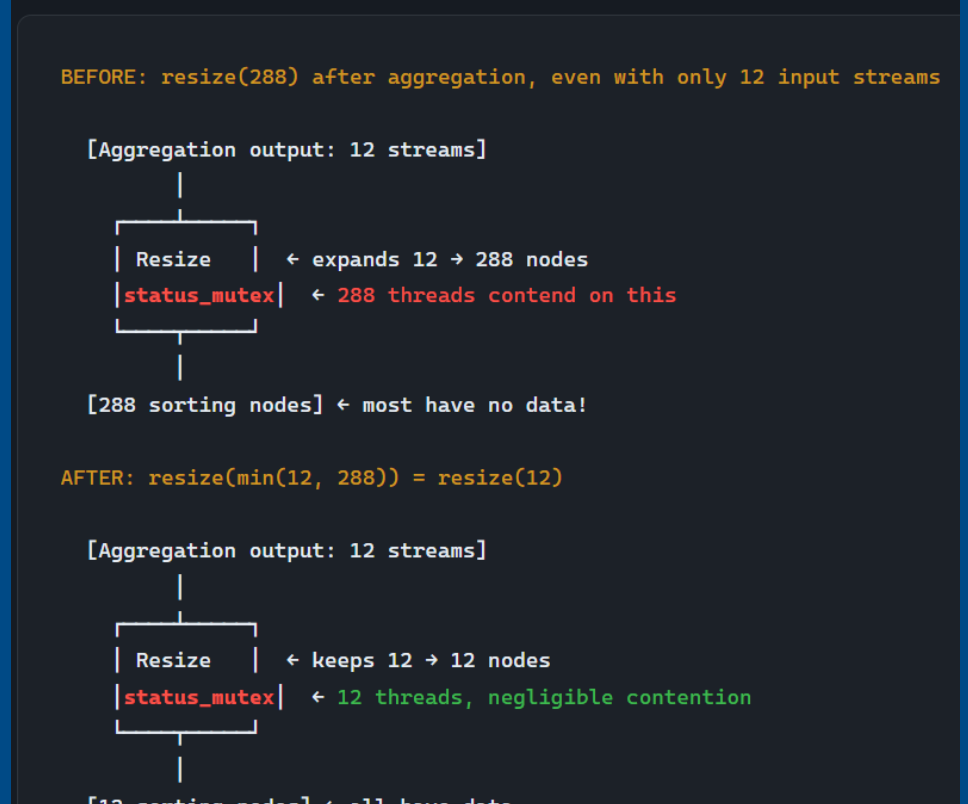
Patch 3: Hierarchical Merging for MergingSortedTransform

0003-add-hierarchical-merging-for-MergingSortedTransform

3.29x @Q42 on 288c

Patch 4: Cap Post-Aggregation Resize

0004-cap-post-aggregation-resize-to-input-streams



Q42 CWF 288c: Core Scaling

Q42: SELECT DATE_TRUNC('minute', EventTime) AS M, COUNT(*) AS PageViews FROM hits WHERE CounterID = 62 AND EventDate >= '2013-07-14' AND EventDate <= '2013-07-15' AND IsRefresh = 0 AND DontCountHits = 0 GROUP BY DATE_TRUNC('minute', EventTime) ORDER BY DATE_TRUNC('minute', EventTime) LIMIT 10 OFFSET 1000;